

Concurrency Control

Concurrency Control

- Given an execution (schedule) we can test whether the execution was serializable
 - If execution was serializable, then ok
 - If not serializable, then it's too late!
- Concurrency control:
 - during execution take measures such that a non-serializable execution can never happen

Concurrency Control: Strict 2-phase Locking

- No conflict: transactions can execute at the same time
- Upon first conflict: the second transaction has to wait until the first transaction releases the lock
- Locks: Two types, because two read operations do not conflict
- Basics of locking:
 - Each transaction T_i must obtain a S (*shared*) lock on object *before* reading, and an X (*exclusive*) lock on object *before* writing.
 - X lock on object O is granted if no other lock active on O
 - X lock granted $\rightarrow$ no other lock on O
 - S lock on object O is granted if no other lock or only S locks are granted
 - S lock granted $\rightarrow$ no X lock on O
 - If a conflicting lock is active, the transaction must wait until the lock is released
 - Conflicting locks are expressed by the compatibility matrix:

	S	X
S	✓	--
X	--	--

Strict Two Phase Locking

- Phase 1 = growing phase: acquiring locks whenever you need one
- Phase 2 = shrinking phase: After a transaction has released one of its lock (unlock) it may not request any further locks
- **Strict:** a transactions releases all its lock at the end of its execution after commit
 - Shrinking phase happens in one shot at end of transaction.

Strict 2PL: an Example

T1: $R(A), W(B)$

T2: $W(A)$

T3: $R(A), R(B)$

Order of submission: R1(A), R3(A), W1(B), W2(A), R3(B)

Lock Table: empty at start

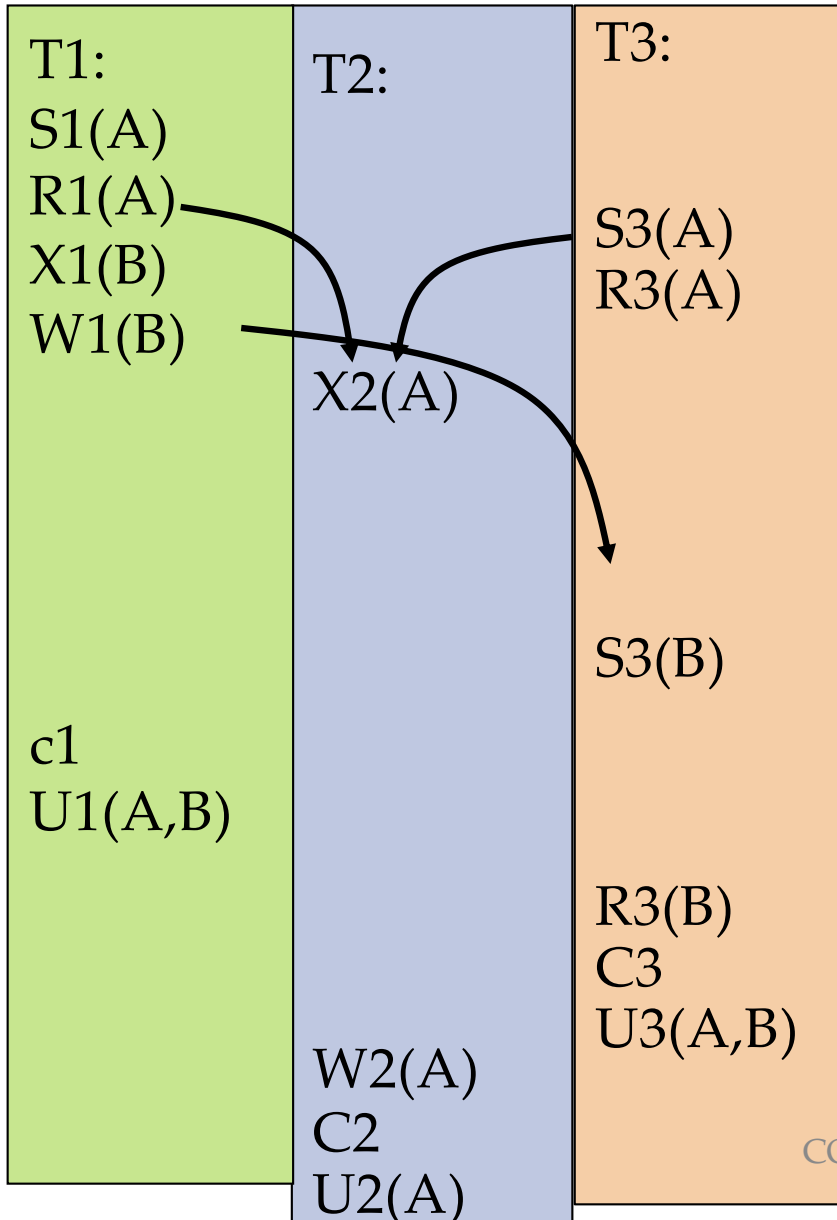


Diagram illustrating the execution of a program with two threads, A and B, on a multiprocessor system. The diagram shows the sequence of instructions and the state of the cache (T1, T2, T3) for each thread.

Thread A's instructions and cache state:

- A: T1:S (Cache: T1:S)
- A: T1:S T3:S (Cache: T1:S, T3:S)
- A: T1:S T3:S (Cache: T1:S, T3:S)
- A: T1:S T3:S (Cache: T1:S, T3:S)
- WL T2:X (Cache: T2:X)
- A: T1:S T3:S (Cache: T1:S, T3:S)
- WL T2:X (Cache: T2:X)
- A: T3:S (Cache: T3:S)
- WL T2:X (Cache: T2:X)
- A: T2:X (Cache: T2:X)

Thread B's instructions and cache state:

- B: T1:X (Cache: T1:X)
- B: T1:X (Cache: T1:X)
- B: T1:X (Cache: T1:X)
- WL T3:S (Cache: T3:S)
- B: T3:S (Cache: T3:S)

The diagram shows that Thread A's execution is interrupted by Thread B's execution, leading to a cache coherence issue where Thread A's cache (T1:S) contains a stale value of T1, while Thread B's cache (T1:X) contains a newer value of T1. This results in Thread A's final output (T2:X) being based on stale data.

COMP 421/764 © McGill

Implementing strict 2PL

- Lock request
 - If lock is S, no X lock is active and the request queue is empty:
 - add the lock to the granted lock queue and set lock type to S
 - If lock is X and no lock active ($\Rightarrow$ the request queue is also empty):
 - add the lock to the granted lock queue and set lock type to X
 - Otherwise:
 - add the lock to the request lock queue
 - In the first two cases, the transaction can continue immediately. In the last case the transaction is blocked until the lock is granted
- Lock release (at end of transaction)
 - Remove the lock from the granted lock queue
 - If this was the only lock granted on the object:
 - grant one write lock (if the first lock in the request queue is a write) or
 - n read locks (if the first n locks in the request queue are reads) as described above.

Details

- A transaction does not request the same lock twice.
- A transaction does not need to request a S lock on an object for which it already holds an X lock.
- If a transaction has an S lock and needs an X lock (upgrade from S to X) it must wait until all other S locks (except its own) are released, and then its S lock is transformed into a X lock.
 - Note that the X lock is granted even if others are waiting for an X lock (to avoid deadlock – see following slides)

Implementation Details

- Locks are managed using a **lock table**
- The lock table has a lock table entry for each object that is currently locked
 - Pointer to queue of granted locks (or simply the number of transactions currently holding a lock)
 - Type of lock held (shared or exclusive)
 - Pointer to queue of lock requests (waiting transactions)
 - A transaction T contains only one lock per object
 - if a T has an S lock and requests an X lock, the S lock is upgraded to an X lock (may have to wait/block for other S locks to be released)
- Locking and unlocking have to be atomic operations
 - Set latch/semaphore when accessing lock table
- Transaction table:
 - For each transaction T contains pointer to a list of locks held by T

Why does 2PL work?

- When is a schedule not serializable?
 - If there are operations of transactions T1 and T2 such that T1 should be ordered before T2 AND after T2 in the schedule $T1 \rightarrow T2 \rightarrow T1$

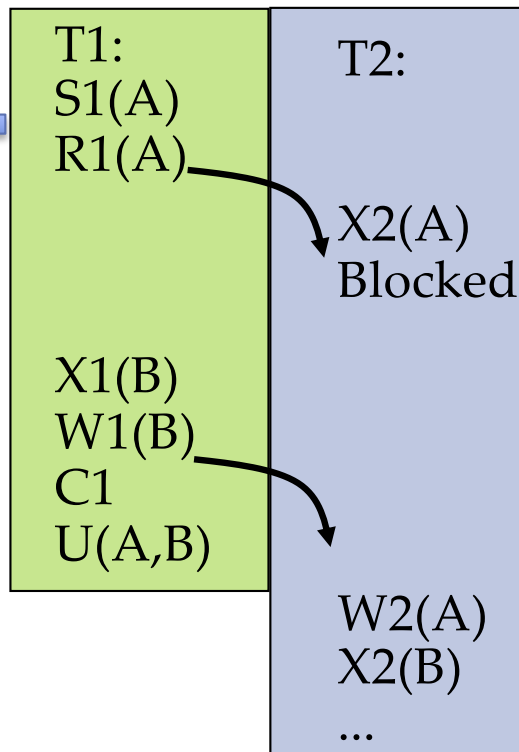
Submission order 1

$R1(A)$ $W2(A)$ $W2(B)$ $W1(B)$

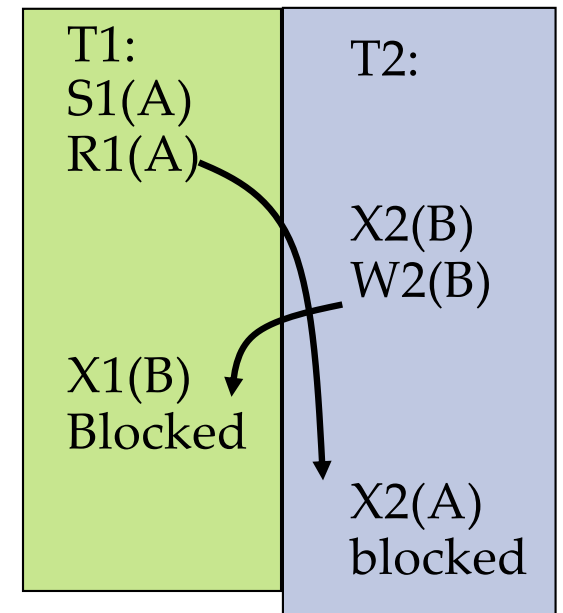
Submission order 2

$R1(A)$ $W2(B)$ $W1(B)$ $W2(A)$

$R1/W2 + R2/W1$
 $R1/W2 + W2/W1$
 $R1/W2 + W2/R1$
 $W1/W2 + R2/W1$
 $W1/W2 + W2/W1$
 ...
 $W1/R2 + R2/W1$
 ...



transactions ordered according
to who got the first lock



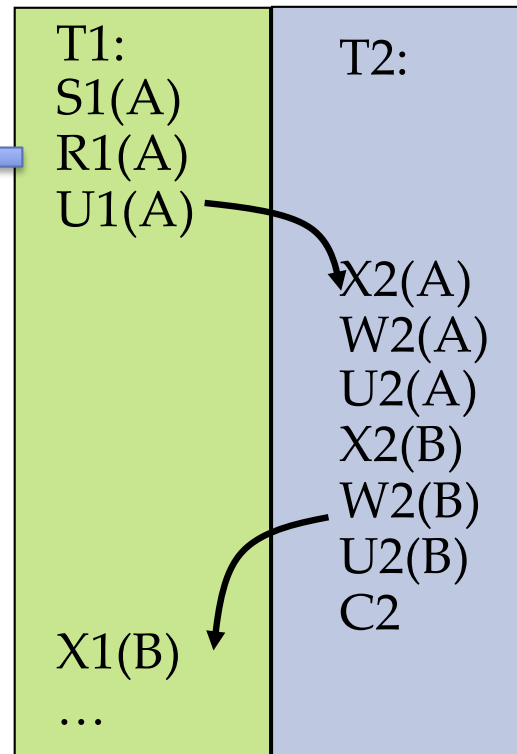
Deadlock:
see later

Why does simple locking NOT work?

- It allows any order!!

Submission order 1
 $R1(A)$ $W2(A)$ $W2(B)$ $W1(B)$

$R1/W2 + R2/W1$
 $R1/W2 + W2/W1$
 $R1/W2 + W2/R1$
 $W1/W2 + R2/W1$
 $W1/W2 + W2/W1$
...
 $W1/R2 + R2/W1$
...

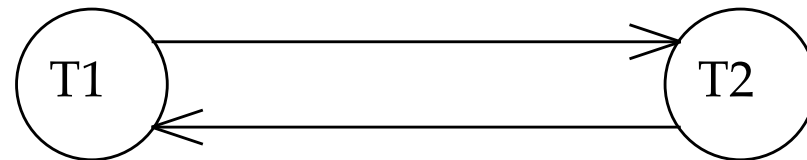
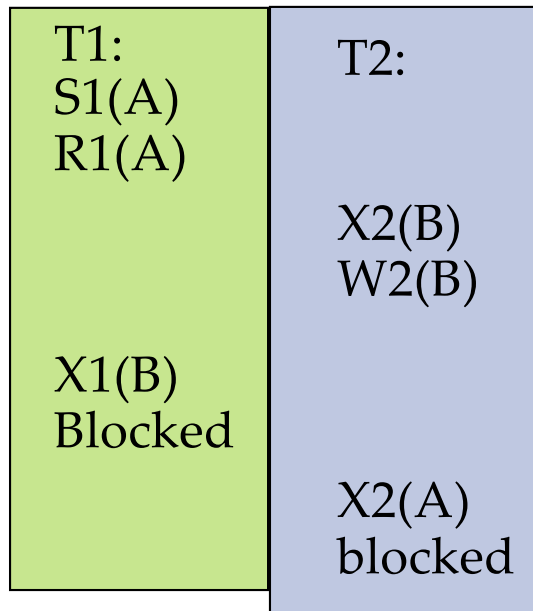


Non serializable!

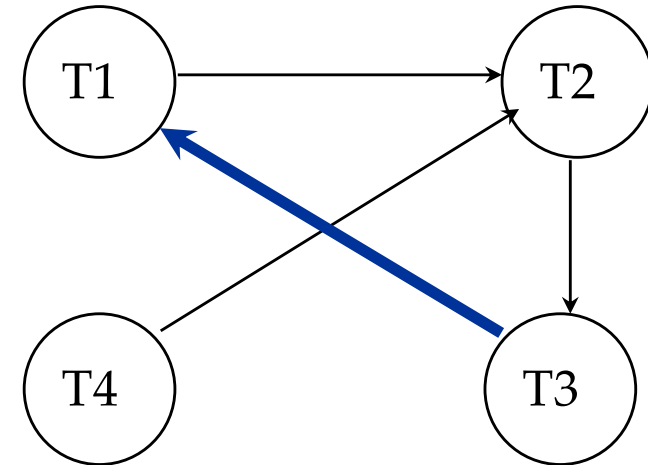
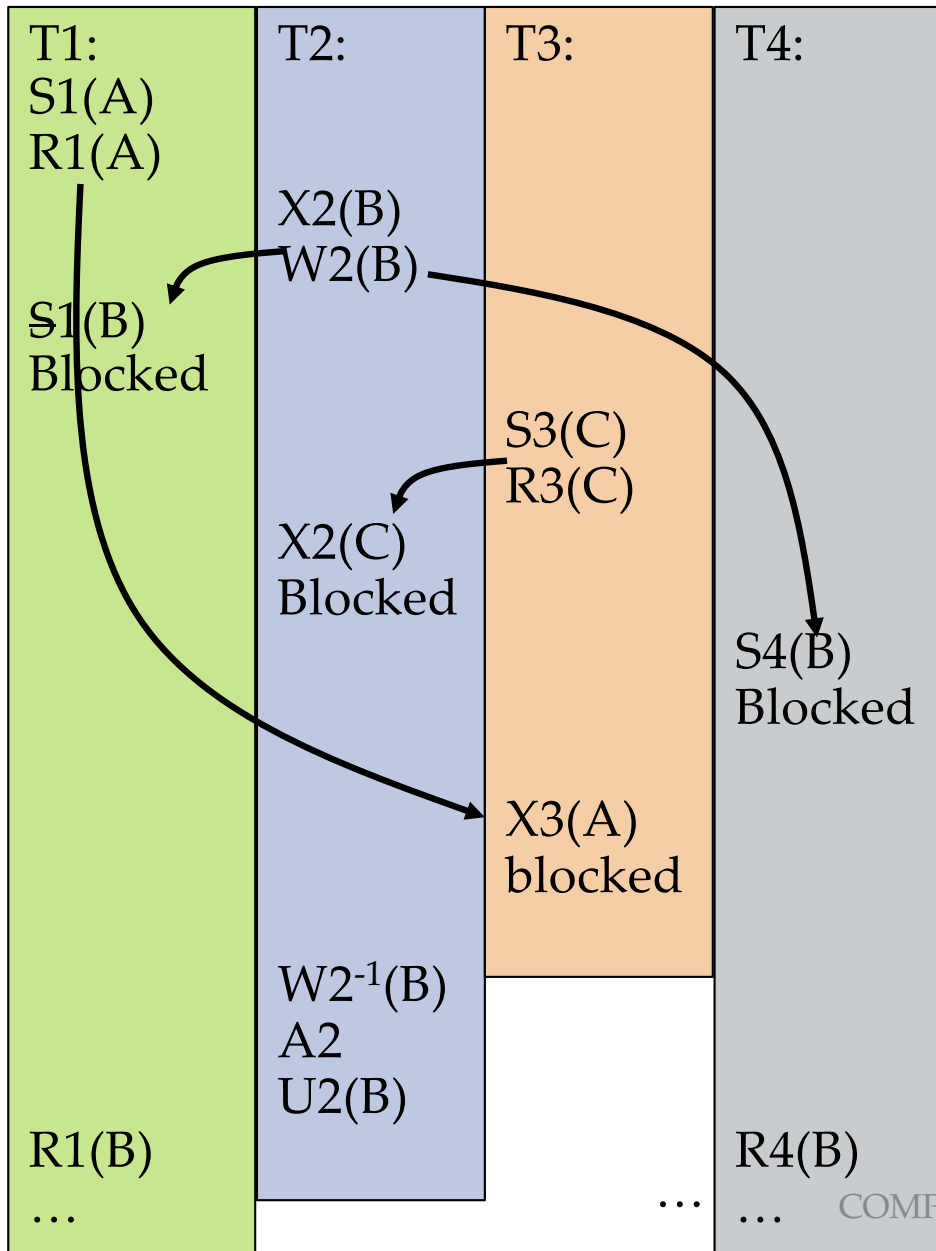
Operations are ordered as they
are submitted

Deadlocks

- Deadlock: Cycle of transactions waiting for locks to be released by each other.
- Waits-for graph:
 - Nodes are transactions
 - There is an edge from T_i to T_j if T_i is waiting for T_j to release a lock
- Deadlock detection: look for cycles in the wait-for graph

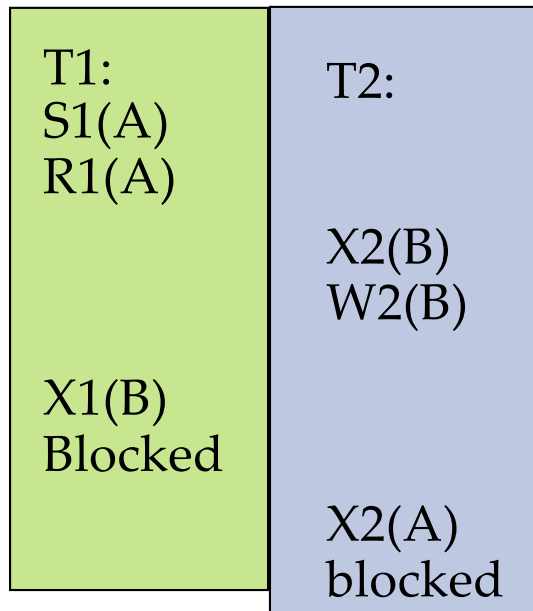


Deadlock Detection (Continued)

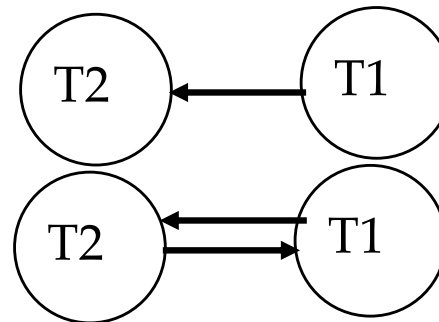


Dependency graph - wait-for-graph

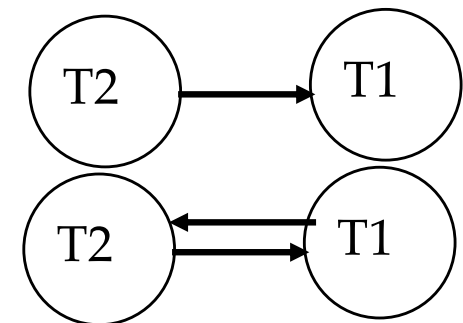
- Note: is similar to dependency graph with the following difference
 - If there is an edge from T2 to T1 in the wait-for-graph, then T2's operation will execute after T1's operation (T2 waits for T1 to release the lock), hence, in the dependency graph there is an edge from T1 to T2
 - Deadlocks can happen because 2PL avoids unserializable schedules by locking objects!



Wait-for-graph



Depend. graph



Deadlock (contd).

- Timeout Mechanism
 - if a transaction waits for a lock longer than a predefined timeout interval, assume it is in a deadlock cycle and abort the transaction
 - Disadvantage: choice of adequate timeout interval is crucial
- Alternative Prevention: Conservative 2PL
 - Request all locks at begin of transaction

Challenges in a real DBMS

- What's your data item?
 - If a relation in the database
 - Not much concurrency!
 - If a tuple in a database – Phantom problem
 - T1: SELECT max(age) FROM Skaters WHERE rating = 5
 - T1 has S lock on all existing tuples with rating = 5
 - T2: INSERT INTO Skaters (sid, rating, age) VALUES (11, 5, 15) /
 - T2: INSERT INTO Skaters (sid, rating, age) VALUES (12, 6, 29)
 - * T2 has X lock on two new tuples
 - T2 commits and releases all locks
 - T1: SELECT max(age) FROM Skaters WHERE rating = 6
 - T1 has S on all tuples with rating = 6 (including the one inserted by T2)
 - T1 reads one but not both tuples created by T2

Skaters

<u>sid</u>	sname	rating	age	
28	yuppy	5	10	S – T1
31	debby	6	10	S – T1
11		5	15	X – T2
12		6	29	X – T2

Isolation Levels

Isolation Level/ Anomaly	Dirty Read	Unrepeatable Read	Phantom
Read Uncommitted	maybe	maybe	maybe
Read Committed	no	maybe	maybe
Repeatable Read	no	no	maybe
Serializable	no	no	no

- Implementation via Locking:
 - Exclusive locks are always held until end of transaction; -- always avoid dirty writes;
 - differences in shared locks
- Read Uncommitted
 - Read op. do not set locks; can read not-committed updates
- Read Committed
 - Read op. set short S locks; have to wait for X locks to be released
 - release shared lock immediately after execution of op.
- Repeatable Reads
 - Read operations set standard lock S locks; standard 2PL on a tuple basis
 - Serializable if no predicate reads/writes (no complex where clauses)
- Serializable
 - Read op. must set S locks that cover all objects that are read
 - predicate locks or coarse locks (e.g., lock on entire relation)

Other Concurrency Control Mechanism

- Many existing system implement locking-based approaches
 - With strict 2PL the most restrictive
 - **Snapshot Isolation**: Oracle, PostgreSQL, MicrosoftSQL server
 - Example of a **multi-version** system
 - Very attractive in distributed database systems
 - Optimistic concurrency control